Job Automation and the Future of Work: Predicting Job Automation Risk and Assessing Its Relationship with Skills and Employment Trends

A Final Technical Report Submitted in Partial Fulfillment of the Requirements for DS 5500: Capstone — Applications in Data Science

Authors: Elif-Selma Yilmaz
Rayan Hassan
Team: Team 13
Instructor: Dr. Fatema Nafa
Course: DS 5500 — Capstone: Applications in Data Science
Institution: Northeastern University, Khoury College of Computer Sciences

Date: April 7, 2026
GitHub Repo: https://github.com/selmayilmaz13/DSCapstone

Abstract

Background: Advances in artificial intelligence and automation are increasingly changing how work is performed across industries. Understanding automation exposure at the occupation level matters because it affects how people think about job stability, future demand, and the broader labor market.

Objective: The goal of this project is to build a system that estimates the automation risk of a given job title while also providing employment outlook and skill-based context.

Methods: We combined a supervised machine learning pipeline with an LLM-based estimate and a rule-based baseline. We feature-engineered occupation-level skills from O*NET-based data and merged them with an external automation-risk dataset using SOC code and job-title matching to form our training data. We then trained a Random Forest regressor which is used as the main predictive model. On the other hand, we prompt-engineered an OpenAI LLM Assistant, which sits on AWS, to give us its predictions based on a formatted input that includes job matches and skill information from the O*NET dataset.

Results: The Random Forest regressor outperformed a linear baseline on the test set, showing stronger overall fit and lower prediction error. The LLM predictions were close to our ML model for the most part, showing satisfactory AI reasoning. The final system was deployed as an interactive Streamlit app that combines ML prediction, LLM estimation, a rule-based index, employment outlook, and skill summaries in one interface.

Conclusion: The project shows that combining structured occupation-level modeling with LLM-based reasoning can provide a more complete view of automation risk than relying on a single method. The final system offers a practical and interpretable way to explore automation exposure for user-entered job titles.

Keywords: Automation Risk, Labor Market, Random Forest, OpenAI LLM, O*NET Skills

Contents

List of Figures

List of Tables

1. Introduction

1.1 Problem Statement

Artificial intelligence and automation are changing how work is carried out across many industries. Some occupations are more exposed to automation than others, but that exposure is not always easy to estimate in a structured way. Job titles alone can be vague, and many existing discussions of automation risk rely on static occupation-level scores that do not easily handle flexible user input.

The goal of this project is to build a system that estimates the automation risk of a given job title while also providing employment outlook and skill-based context. Instead of relying on a single method, the system combines a supervised machine learning model, an LLM-based estimate, and a rule-based baseline. This allows the project to compare different ways of reasoning about automation exposure and to present the results in a more interpretable way.

A practical challenge in this problem is that users often enter job titles in non-standard or ambiguous forms. To address this, the system first maps a user-provided title to the closest standardized occupation. It then combines structured occupation data, learned predictions, and reasoning-based estimates to generate the final output.

1.2 Motivation and Significance

This problem is important because automation exposure affects how people think about job stability, future demand, and changing skill requirements. A tool that gives a clearer view of automation risk can be useful both for individual users and for broader labor market analysis.

This project is also motivated by the limitations of relying on a single approach. An LLM can provide flexible reasoning, but its outputs may be inconsistent. A rule-based score is easy to interpret, but it depends on manual design choices. A supervised machine learning model is more grounded in labeled data, but it still depends on the quality of the feature engineering and target dataset. By comparing these approaches in one system, the project aims to provide a more balanced view of job automation risk.

1.3 Research Questions / Objectives

RQ1. Can occupation-level features engineered from O*NET-based data be used to predict automation risk through supervised learning?

RQ2. How does a trained machine learning model compare with an LLM-based estimate and a rule-based baseline for the same task?

RQ3. Can these components be integrated into a single interactive system that allows users to explore automation risk, employment outlook, and skill-based context for a given job title?

1.4 Report Structure

This report is organized as follows: Section 2 reviews related work; Section 3 describes the data pipeline and preprocessing steps; Section 4 presents model selection and implementation; Section 5 reports the evaluation results; Section 6 summarizes testing and validation; Section 7 documents GitHub usage and team contributions; Section 8 discusses deployment and reproducibility; and Section 10 concludes with limitations and future work.

2. Related Work

Research on automation risk and AI exposure has developed along several related directions. Some work measures how exposed occupations are to AI capabilities, while other work studies automation risk through task composition or examines how AI affects labor demand. Our project builds on these ideas, but differs in that it combines a supervised machine learning model, an LLM-based estimate, and a rule-based baseline inside a single interactive system.

2.1 Task and Occupation Exposure to AI

Recent work has studied how AI systems, especially large language models, may affect work tasks across occupations. Eloundou et al. [2024] analyze task exposure across standardized occupations and show that a substantial share of workers may have at least some tasks exposed to LLM capabilities. Their work provides a useful occupation-level view of LLM exposure, but the outputs remain tied to predefined occupations.

A related perspective is provided by Felten et al. [2021], who propose the AI Occupational Exposure index using O*NET-based information. Their results show that AI exposure varies across occupations and is strongly related to task composition. Together, these studies motivate our use of structured occupation-level information, but our system extends them by handling flexible user-entered job titles rather than only static occupation categories.

2.2 Automation Risk as a Task-Based Problem

Arntz et al. [2016] argue that automation risk should be understood through task composition rather than job titles alone. Their results show that workers within the same occupation may face different levels of automation exposure depending on the tasks they actually perform. This insight

is central to our project. We adopt the same general idea, but operationalize it through engineered occupation-level features and AI-based reasoning instead of survey-based task categories alone.

2.3 Labor Demand and Employment Context

Acemoglu et al. [2022] study how AI adoption affects hiring patterns using online vacancy data. Their findings suggest that AI can shift demand away from some traditional skills while increasing demand for others. Although their work does not estimate automation probabilities directly, it motivates our decision to include employment outlook alongside automation risk so that users can view exposure together with projected demand trends.

2.4 Positioning of This Work

Prior work often provides static occupation-level exposure measures or focuses on predefined occupational categories. In contrast, our system is designed to accept flexible user-entered job titles, map them to standardized occupations, and then combine multiple views of automation risk. The main predictive component is a supervised machine learning model trained on occupation-level features engineered from O*NET-based data and an external automation-risk dataset. Alongside this model, we include an LLM-based estimate and a rule-based baseline to provide comparison and interpretability.

3. Data Pipeline and Preprocessing

The final system relies on a data pipeline that combines structured occupation-level information with an external automation-risk target. The goal of this pipeline was to transform raw occupation and skill data into a model-ready feature table and then align it with a labeled automation-risk dataset for supervised learning.

3.1 Dataset Description

Table 1: Dataset summary

Dataset	Source	Samples	Role
O*NET-based occupation dataset	O*NET / BLS	800+ occupations	Feature engineering using skills, task-related values, and employment projections
External automation-risk dataset	External source	600+ occupations before merge	Supervised learning target using occupation-level automation probabilities

The project uses two main data sources. The first is an O*NET-based occupation dataset containing occupation titles, SOC-related identifiers, employment projections, skill groups, O*NET element names, and O*NET data values. This dataset serves as the main source of structured explanatory features.

The second is an external automation-risk dataset used as the supervised learning target. It provides occupation-level automation probability values together with occupation identifiers that can be aligned with the O*NET-based data.

After preprocessing and merging, the final supervised training table contained 402 matched occupations and 131 usable numeric feature columns.

3.2 Exploratory Data Analysis (EDA)

Exploratory analysis focused on two questions: how the raw occupation data was structured and whether skill composition showed meaningful relationships with labor market outcomes. The raw skills dataset contained 144,352 rows and 13 columns, which confirmed that the data was stored in long format rather than as one row per occupation.

An initial occupation-level skill matrix was created by pivoting O*NET element values into wide-format features. In early analysis, this produced 832 occupations and 103 skill columns. This step showed that the raw data could be transformed into a structured feature table suitable for modeling.

EDA also suggested that occupations with stronger projected employment growth placed greater emphasis on communication, science, service interaction, and systems-oriented skills, while shrinking occupations were more associated with manual, mechanical, and control-oriented tasks. These patterns supported the decision to engineer occupation-level skill features for the final supervised learning pipeline.

The figure below highlights the skill groups that differ most strongly between high-growth and low-growth occupations. Skills related to communication, public interaction, science, service, and systems-oriented work appear more strongly in occupations with higher projected employment growth.

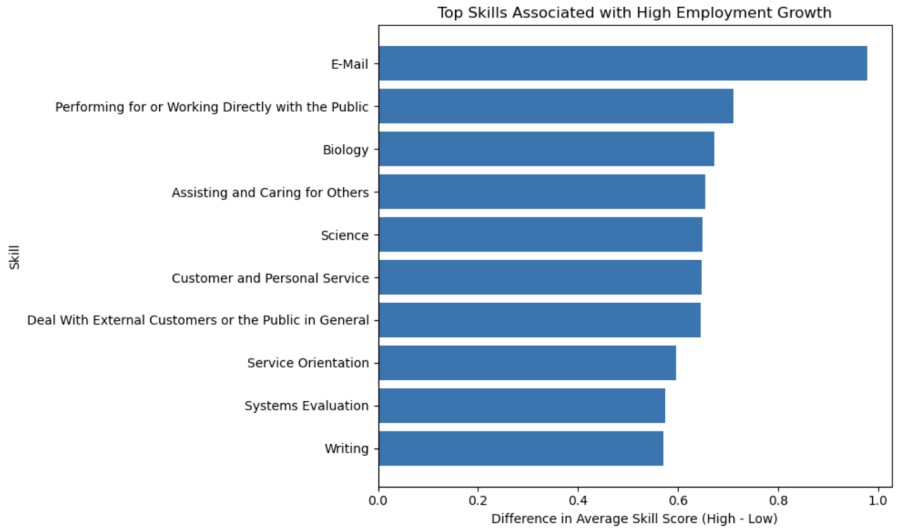

Figure 1: Top skills associated with high employment growth

3.3 Data Cleaning and Preprocessing

After the exploratory stage, the raw occupation data was cleaned and standardized for downstream modeling. Column names were normalized to a consistent format, and only the fields relevant to occupation identity, employment projections, and O*NET-derived skill values were retained.

The external automation-risk dataset was prepared separately and then aligned with the occupation-level feature table. SOC code was used as the primary merge key because it provided the most reliable standardized occupation identifier. When direct code matches were unavailable, job-title matching was used as a fallback.

3.3.1 Missing Value Strategy

Missing values were handled mainly during the construction of the occupation-level feature table. After pivoting O*NET elements and skill groups into wide-format columns, some features had incomplete coverage across occupations. Columns that were entirely missing or unusable for modeling were removed. For the supervised learning pipeline, the remaining numeric missing values were handled through median imputation inside the scikit-learn pipeline.

3.3.2 Outlier Handling

Outlier handling was not a major focus of the final pipeline. Most engineered features were occupation-level averages or bounded O*NET-related values, which are less prone to extreme noise than transactional data. As a result, no aggressive outlier removal procedure was applied. Instead, the modeling strategy relied on robust feature construction and a tree-based model, which is less sensitive to outliers than purely linear approaches.

An additional AI job trends dataset was explored earlier in the project, but it was not included in the final pipeline because it did not align cleanly with the occupation-level feature representation used in the supervised learning setup.

3.4 Feature Engineering and Transformation

Feature engineering was one of the central parts of the project. The O*NET-based data was transformed from long-format records into occupation-level numeric features that could be used for supervised learning.

Employment-related variables such as employment in 2024, projected employment in 2034, and employment change fields were retained directly. O*NET element values and skill-related attributes were then aggregated at the occupation level, and both skill groups and element names were pivoted into wide-format numeric columns so that each occupation could be represented as a single structured feature vector.

This process produced a high-dimensional tabular representation of occupations in which each row corresponded to one occupation and each column captured a different skill, task-related value, or employment-related attribute. In the final supervised training setup, this resulted in 131 usable numeric feature columns.

In addition to the machine learning features, we created a separate rule-based baseline using selected O*NET-derived variables. Certain traits were manually treated as stronger indicators of automation exposure, while others were treated as indicators of lower exposure. These normalized components were combined into a simple rule-based risk index that was later used in the application as a comparison baseline.

3.5 Train / Validation / Test Split

For the supervised learning stage, the merged dataset was split into training and test sets using an 80/20 split. Because the dataset size was moderate, the focus in this project was on establishing a reliable held-out evaluation rather than building a separate large validation framework. Model comparison was performed on the same test split using a linear baseline and a Random Forest

regressor.

Table 2: Data partition sizes

Split	Samples	Percentage	Notes
Training	321	80%	Used for model fitting
Test	81	20%	Held-out evaluation set

3.6 Pipeline Pseudo-code

Pseudo-code: End-to-End Occupation Data Preparation Pipeline

Require: O*NET-based occupation dataset $\mathcal{D}_{onet}$, external automation-risk dataset $\mathcal{D}_{risk}$

Ensure: Final merged dataset $(\mathcal{D}_{train}, \mathcal{D}_{test})$ for supervised learning

Load $\mathcal{D}_{onet}$ and $\mathcal{D}_{risk}$ from source files
Normalize column names and occupation identifiers
Select relevant fields from $\mathcal{D}_{onet}$
Aggregate long-format O*NET records into one row per occupation
Pivot skill groups and O*NET element names into wide-format numeric columns
Retain employment-related variables as additional numeric features
Remove columns that are entirely missing or unusable for modeling
Merge engineered occupation-level table with $\mathcal{D}_{risk}$ using SOC code
if no direct SOC code match is found **then**
    Apply job-title matching as a fallback
end if
Separate target variable y from feature matrix X
Split merged dataset into training and test sets using an 80/20 split
Save processed feature table and training artifacts for downstream modeling
return $(\mathcal{D}_{train}, \mathcal{D}_{test})$

4. Model Selection and Implementation

4.1 Model Family Justification

Table 3: Model family comparison for this task

Family	Strengths	Limitations	Selected?
Classical ML	Works well on structured tabular data, interpretable baseline, lower compute cost	May miss more complex patterns if the feature space is weak	✓
Deep Learning	Can model highly complex relationships and learn representations automatically	Requires larger datasets, more tuning, and higher compute cost	—
LLM / Foundation Models	Flexible with user-entered job titles, useful for reasoning and semantic matching	Outputs may be inconsistent, expensive at inference time, not ideal as the only predictive model	✓

The final system combines two main modeling directions: a supervised machine learning model trained on structured occupation-level features and an LLM-based component used for job-title matching and comparison. This choice was driven by the nature of the task and the available data. Once the occupation data had been transformed into a tabular feature space and merged with an external automation-risk target, the problem became a supervised regression task on a moderate-sized structured dataset. Under these conditions, classical machine learning was a more appropriate fit than deep learning.

Within the supervised learning family, we compared a simple linear baseline with a tree-based ensemble model. Linear Regression was useful as an initial benchmark because it provided a straightforward reference point. However, the engineered occupation-level features were high-dimensional and likely contained nonlinear relationships, which made a Random Forest regressor more suitable as the main model. At the same time, the LLM-based component was retained because the project initially relied on language-model reasoning before a suitable target dataset was found. Rather than treating the LLM as the main predictive model, we used it as a comparison point alongside a rule-based baseline, while the Random Forest served as the primary learned predictor.

4.2 Model Architecture

The final predictive setup centers on a Random Forest regressor trained on occupation-level numeric features engineered from O*NET-based data. Each row in the feature table corresponds to one

occupation, and each column represents a skill-related, task-related, or employment-related attribute. The target variable is an occupation-level automation probability obtained from the external automation-risk dataset.

The deployed system also includes an LLM-based estimate generated through the OpenAI API and a rule-based baseline constructed from selected O*NET-derived features. These are not part of the Random Forest model itself, but are used alongside it to compare different views of automation risk within the application.

Figure 2 summarizes the overall system architecture. It shows the deployed application flow, including the Streamlit frontend, AWS backend, OpenAI integration, and local machine learning inference, together with the training pipeline used to build the saved Random Forest model.

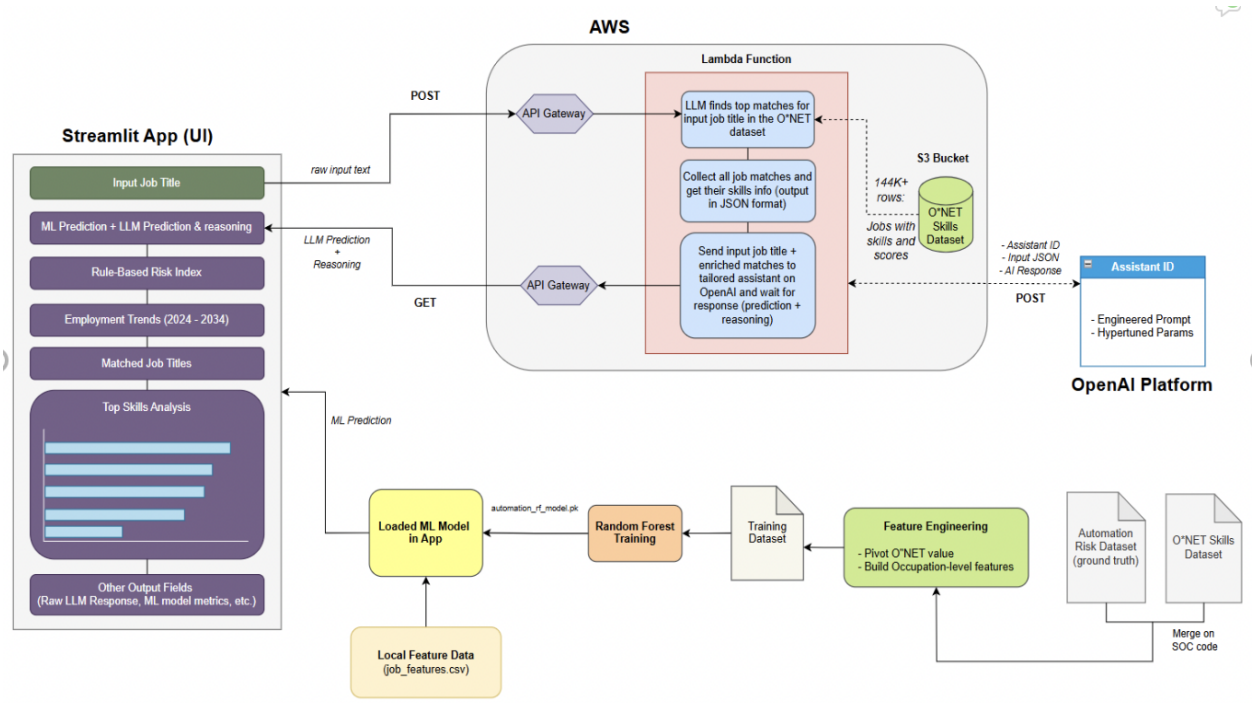

Figure 2: System architecture

4.3 Coding Paradigm and Design Architecture

4.3.1 Paradigm Choice

The project follows a modular and mostly procedural design. Rather than building the system around a large object-oriented codebase, the implementation is divided into separate scripts that each handle a specific stage of the pipeline, such as feature engineering, dataset merging, model training, rule-based scoring, backend processing, and frontend interaction. This was a good fit for the project because the overall workflow is naturally organized as a sequence of stages.

The codebase also separates offline model-building from runtime application logic. Data preparation and feature construction are handled independently from model training, which makes it possible to save the trained model and reuse it later without retraining. The frontend, backend, and local machine learning inference are also kept separate, which made the system easier to debug, extend, and deploy.

4.3.2 Module Structure

```
DSCapstone/
|-- Code/
|   |-- app.py                      # Streamlit frontend
|   |-- aws_lambda.py               # AWS Lambda backend logic
|   |-- feature_engineering.py      # Occupation-level feature creation
|   |-- merge_training_data.py      # Merge engineered features with
    automation target
|   |-- model_training.py           # Train and evaluate ML model
|   |-- predict_model.py            # Local model inference
|   |-- scoring.py                  # Rule-based baseline
|-- Datasets/
|   |-- automation_rf_model.pkl     # Saved Random Forest model
|   |-- job_features.csv            # Occupation-level feature table
|   |-- job_features_scored.csv     # Scored occupation-level features
|   |-- training_dataset.csv        # Final merged training table
|   |-- model_predictions.csv       # Saved predictions
|   |-- rf_feature_importances.csv # Feature importance outputs
|-- Notebooks/
|   |-- NN_Automation_Risk_Analysis.ipynb
|   |-- Skill_Assessment.ipynb
|-- Reports/
|-- requirements.txt
|-- README.md
```

Listing 1: Repository directory structure

4.4 Hyperparameters

The main supervised model in the final system was a Random Forest regressor implemented inside a scikit-learn pipeline with median imputation. Hyperparameter tuning in this project was limited and primarily manual, since the main objective was to establish a reliable end-to-end pipeline and compare the Random Forest against a simpler linear baseline.

Table 4: Random Forest hyperparameter configuration

Parameter	Value	Selection Method
Number of estimators	300	Manual selection
Max depth	None	Default / manual selection
Random state	42	Fixed for reproducibility
n_jobs	-1	Parallel processing across all available CPU cores

A linear regression model was also trained as a baseline using the same feature table and train/test split. The final model choice was based on comparative test performance rather than extensive automated hyperparameter search.

4.5 Training Procedure Pseudo-code

Pseudo-code: Random Forest Training Pipeline

Require: Merged dataset $\mathcal{D}$ with features X and target y

Ensure: Trained model pipeline $\mathcal{M}^*$

Load $\mathcal{D}$ from disk
Remove rows with missing target values
Select numeric feature columns
Exclude identifier and label columns
Remove columns with no observed values
Split (X, y) into training and test sets using an 80/20 split
Define baseline pipeline:
    median imputation $\rightarrow$ Linear Regression
Define Random Forest pipeline:
    median imputation $\rightarrow$ Random Forest Regressor
Fit baseline pipeline on training data
Predict on test data
Compute MAE, RMSE, and R^2
Fit Random Forest pipeline on training data
Predict on test data
Compute MAE, RMSE, and R^2
Extract feature importances from Random Forest
Save trained pipeline, predictions, and feature importances
return $\mathcal{M}^*$

4.6 Fine-tuning / Prompt Engineering

Fine-tuning was not used in this project. The LLM-based component was implemented through prompt-based interaction with the OpenAI API using `gpt-4.1-mini`.

Three prompt types were used in the AWS backend. The first mapped a user-entered job title to between one and five official occupations from the employment dataset. The second generated a short user-facing explanation based on the matched occupation's rule-based risk result and projected employment outlook. The third produced a standalone LLM-only automation estimate, returning a structured JSON object with an automation-risk score, a risk label, and a short reasoning field.

The prompts were written in an instruction-based, zero-shot style. Whenever possible, they were designed to return structured outputs so that the backend could parse them reliably and pass them to the frontend. In the final system, the LLM supported job-title matching, explanation generation, and comparison, while the main predictive model remained the supervised Random Forest regressor.

Table 5: LLM prompt configuration summary

Prompt Type	**Purpose**	**Model**	**Output**
Job title matching	Match a user-entered title to 1–5 official occupations	`gpt-4.1-mini`	JSON
Explanation generation	Produce a short explanation using the rule-based score and employment outlook	`gpt-4.1-mini`	Text
LLM-only automation estimate	Generate a standalone risk score, label, and short reasoning	`gpt-4.1-mini`	JSON

Table 6: LLM parameter summary

Setting	**Value**	**Notes**
Model	`gpt-4.1-mini`	Used for all backend prompts
Prompting style	Zero-shot, instruction-based	No in-context examples were provided
Temperature	Default	Not explicitly specified in code
Max tokens	Default	Not explicitly specified in code
Structured output	JSON when required	Used for job matching and LLM-only estimate
Backend integration	AWS Lambda	Parsed by backend and returned to frontend

Representative prompt snippets are shown below.

```
You are an expert in job classification.
A user entered this job title: "{input_title}"
Your task is to select the occupations that best match the user's job.

Rules:
- Return between 1 and 5 matches.
- Only choose occupations that truly represent the user's job.
- Ignore occupations that are clearly unrelated.

Return ONLY valid JSON:
{
  "matches": ["job title 1", "job title 2"]
}
```

Listing 2: Prompt snippet: job-title matching

```
You are giving an LLM-only automation risk estimate for a job.
User input job title: {job_title}
Closest official occupation match: {matched_title}

Return only valid JSON in this exact format:
{
  "automation_risk_score": 0 to 100,
  "automation_risk_label": "Low" or "Medium" or "High",
  "reasoning": "2-3 short sentences"
}
```

Listing 3: Prompt snippet: LLM-only automation estimate

5. Results and Model Evaluation

5.1 Evaluation Metrics

Because the main predictive task in this project is regression, evaluation focused on metrics that measure continuous prediction error and overall model fit. We used Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2) to compare the Random Forest regressor against a linear baseline on the held-out test set.

MAE was used because it provides an intuitive measure of the average absolute prediction error. RMSE was included because it penalizes larger errors more strongly, which is useful when large deviations are especially undesirable. The coefficient of determination was used to measure how well the model explains variation in the target relative to a simple baseline.

Table 7: Performance metrics and their applicability

Metric	Formula	Why Chosen
MAE	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Measures average absolute prediction error
RMSE	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$	Penalizes larger prediction errors more strongly
R^2	$1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	Measures overall model fit relative to the mean baseline

5.2 Baseline Comparison

The main supervised comparison in this project was between a simple linear baseline and a tree-based ensemble model. Linear Regression was used as an initial reference point, while the Random Forest regressor was evaluated as the primary model for capturing more complex relationships in the engineered occupation-level feature space.

Table 8: Model performance comparison on the test set

Model	MAE	RMSE	R^2
Linear Regression	0.2218	0.2687	0.4630
Random Forest Regressor	**0.1832**	**0.2315**	**0.6012**

The Random Forest regressor outperformed the linear baseline on the held-out test set. It achieved lower absolute error, lower root mean squared error, and a higher coefficient of determination, which supported its selection as the main predictive model in the final system.

To evaluate how well the LLM performs, we selected a set of job titles and compared its predictions to those produced by the ML model, which we use as our reference. To measure the difference between the two, we computed the Mean Absolute Error (MAE). By calculating the absolute difference for each job and averaging across all samples, we find that the LLM predictions deviate from the ML predictions by about ±12

Given that LLM outputs can be inherently variable and sometimes unpredictable, this level of difference was acceptable for our purposes. Rather than relying only on numerical metrics, we found it more meaningful to also evaluate the quality of the predictions using common sense. In practice, this meant considering both the ML prediction as a reference point and the LLM's accompanying reasoning to judge whether its outputs were sensible and well-justified.

5.3 Learning Curves and Training Dynamics

Traditional learning curves over epochs were not applicable in this project because the main supervised model was a Random Forest regressor rather than an iterative deep learning model. The training procedure did not involve epoch-based optimization, validation-loss tracking, or early stopping.

Instead, model comparison was based on held-out test performance after fitting each model on the same training split, rather than on iterative optimization behavior during training.

5.4 Error Analysis

Error analysis in this project focused mainly on model behavior at the occupation level rather than on individual examples in a large-scale instance dataset. Since the target is an occupation-level automation probability, prediction errors are more likely to reflect ambiguity in how automation risk is defined than simple input noise alone.

One limitation is that occupations with broad or mixed task profiles may be harder to model accurately. Jobs that combine cognitive, interpersonal, and routine components do not fit neatly into a single automation-risk pattern, which can make both the machine learning model and the LLM-based estimate less stable. In addition, the external automation-risk target is itself an approximation, so some prediction error likely reflects uncertainty in the labeling process rather than purely model failure.

The comparison between the Random Forest, the LLM-based estimate, and the rule-based baseline was also informative. In some occupations, the three approaches did not fully agree, suggesting that automation risk is sensitive to the modeling assumptions used. This reinforced the value of presenting multiple views of automation risk in the final application rather than relying on a single score alone.

5.5 Ablation Study

A formal ablation study was not conducted in this project. The main comparative analysis focused on model-level performance, specifically the difference between a linear baseline and the final Random Forest regressor on the same occupation-level feature table.

The closest approximation to an ablation-style comparison was the use of multiple system components with different assumptions. However, these were designed primarily as complementary comparison methods within the final application rather than as controlled removals of individual pipeline components.

6. Testing and Validation

6.1 Testing Framework and Strategy

Testing in this project combined lightweight automated testing with functional end-to-end validation. Automated tests were written with `pytest` for core preprocessing and model-training utilities, while broader validation focused on checking that the deployed application behaved correctly from user input to final output.

The automated tests targeted four key parts of the pipeline: feature and target selection, removal of invalid all-missing training columns, successful fitting and prediction of the Random Forest pipeline on a small sample dataset, and evaluation-metric generation through the model evaluation utility. In addition, the deployed system was tested functionally by verifying that job-title requests were processed correctly through AWS Lambda and API Gateway and that the frontend displayed the expected outputs.

6.2 Automated Test Cases

Table 9: Automated test cases for preprocessing and model training

ID	Description	Input	Expected Output	Status
AT-01	Filter rows with missing target values and exclude invalid feature columns	Small synthetic DataFrame	Valid numeric feature matrix and target vector returned	PASS
AT-02	Remove columns that are fully missing in training data	Synthetic training/test feature tables	Invalid columns removed consistently from both splits	PASS
AT-03	Fit Random Forest pipeline and generate predictions	Small synthetic training and test sets	Model fits successfully and returns finite predictions	PASS
AT-04	Evaluate model and return metrics dictionary	Small synthetic training and test sets	Predictions returned and metrics include MAE, RMSE, and R^2	PASS

6.3 Functional Test Cases

Table 10: Representative functional and integration test cases

ID	Description	Input	Expected Output	Status
FT-01	Load merged training dataset and select valid numeric features	*training_dataset.csv*	Feature matrix and target created without missing target rows	PASS
FT-02	Train Random Forest pipeline with median imputation	Training split	Model fits successfully and produces test predictions	PASS
FT-03	Save trained model artifact	Trained pipeline	*automation_rf_model.pkl* written successfully	PASS
FT-04	Backend processes valid job-title request	Example: "Data Scientist"	Structured response returned from AWS backend	PASS
FT-05	Frontend displays full output for valid input	Example: "Data Scientist"	ML estimate, LLM estimate, rule-based index, and outlook displayed	PASS
FT-06	Fallback job-title matching works for imperfect input	Non-standard job title	Closest occupation match returned instead of system failure	PASS

6.4 End-to-End Validation

End-to-end validation was performed by running the deployed Streamlit application and testing representative job-title queries. A successful run required the frontend to accept user input, the backend to return structured job information, the local Random Forest model to generate a prediction, and the final interface to display all outputs coherently.

This type of validation was especially important because the project depends on integration across several components rather than on a single isolated model. In practice, the main validation criterion was that the system could reliably process a user-entered job title and return consistent automation-related outputs without errors.

6.5 Limitations of Testing

The testing process in this project emphasized core pipeline correctness and system integration rather than exhaustive software testing. A larger production-grade system would benefit from broader edge-case coverage, backend mocking, and more extensive automated validation of API behavior. For the scope of this capstone project, however, the combination of automated `pytest`

checks and end-to-end functional validation was sufficient to confirm the correctness of the core modeling and deployment workflow.

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** `https://github.com/selmayilmaz13/DSCapstone`
- **Visibility:** Public
- **Primary Branch:** `main`

The GitHub repository served as the central version-control platform for the project. It stored the codebase, data files used during development, report materials, and documentation for the deployed application.

7.2 Branching Strategy

Version control followed a simple Git workflow centered on the `main` branch. Changes were developed incrementally and pushed regularly as the project evolved from an initial LLM-based prototype into a full pipeline with supervised learning, backend integration, and deployment support.

Because the team was small, collaboration relied mainly on frequent commits, shared updates, and direct coordination rather than on a complex branching and pull-request workflow. Even with this lightweight setup, GitHub played an important role in tracking progress and maintaining a clean final repository.

7.3 Team Contribution Summary

Table 11: Team contribution summary

Member	Primary Responsibilities
Selma	Data preprocessing, feature engineering, supervised model training, model evaluation, automated testing with `pytest`, Streamlit integration, GitHub organization, and final report writing
Rayan	LLM workflow design, AWS Lambda and API Gateway integration, rule-based baseline development, deployment support, system architecture design, GitHub contributions, and final presentation preparation

7.4 Commit History Excerpt

```
858e5f6 Add pytest tests for model training utilities
8a2a445 Remove devcontainer config
da8397b Refine docstrings
8c72754 Update README app link
bac79d4 Update README with final project description and app link
6e1186d Added Dev Container Folder
2572910 Use Streamlit secrets with local env fallback
2891d9a Move notebooks into separate folder
527a828 Remove old aws lambda llm file
77c6d2c Add final job automation insights app and model pipeline
5471790 backend code (aws lambda)
4442641 Adding Detailed Job Automation Analysis
71ad9c1 Moved into the Code folder
ba0ecf1 Fixed
```

Listing 4: Recent commit log

7.5 Pull Request and Code Review Evidence

Table 12: Collaboration workflow summary

Area	Evidence	Status
Version control	Frequent commits documenting modeling, backend, deployment, and repository updates	Completed
Collaboration	Shared repository with coordinated contributions from both team members	Completed
Pull request review	No formal pull-request review workflow was used due to small team size	Not applicable

8. Deployment and Reproducibility

8.1 Environment and Dependencies

The project was developed in Python and relies on a combination of machine learning, data-processing, frontend, and cloud-integration libraries. The main dependencies include pandas and NumPy for data handling, scikit-learn for preprocessing and supervised learning, Streamlit for the frontend application, and the OpenAI API for the LLM-based estimate. The deployed system also uses AWS Lambda and API Gateway for backend processing.

A representative subset of the environment dependencies is shown below.

```
pandas
```

```
numpy
scikit-learn
streamlit
python-dotenv
requests
openai
```

Listing 5: requirements.txt excerpt

8.2 Reproduction Steps

The project can be reproduced by cloning the repository, installing the required dependencies, configuring the required API credentials, and running the Streamlit application.

Pseudo-code: Reproduction Procedure

Require: Git, Python 3.11+, pip, and valid API credentials

Ensure: Reproducible local run of the Job Automation Insights application

1: **git clone** https://github.com/selmayilmaz13/DSCapstone.git
2: **cd** DSCapstone
3: **pip install** -r requirements.txt
4: Add required credentials either through a local `.env` file or Streamlit secrets
5: Ensure the saved model file and feature data are present in the `Datasets/` directory
6: **streamlit run** Code/app.py

Locally, the application reads API configuration from a `.env` file. In deployment, the same credentials are provided securely through Streamlit secrets. This setup allows the same codebase to run both in local development and in the deployed version of the app.

8.3 Demo Application

The final system was deployed as an interactive Streamlit application, available at: `https://job-automation-insights.streamlit.app/`. Users can enter a job title and view the automation-risk outputs, employment outlook, and skill-related summaries in one interface.

The application combines local and cloud-based components. The Random Forest model is loaded locally from a saved pickle file, while AWS Lambda, API Gateway, and the OpenAI API handle the backend job-title processing and LLM-based estimate.

The screenshots below illustrate the deployed application using the input job title *Data Scientist*.

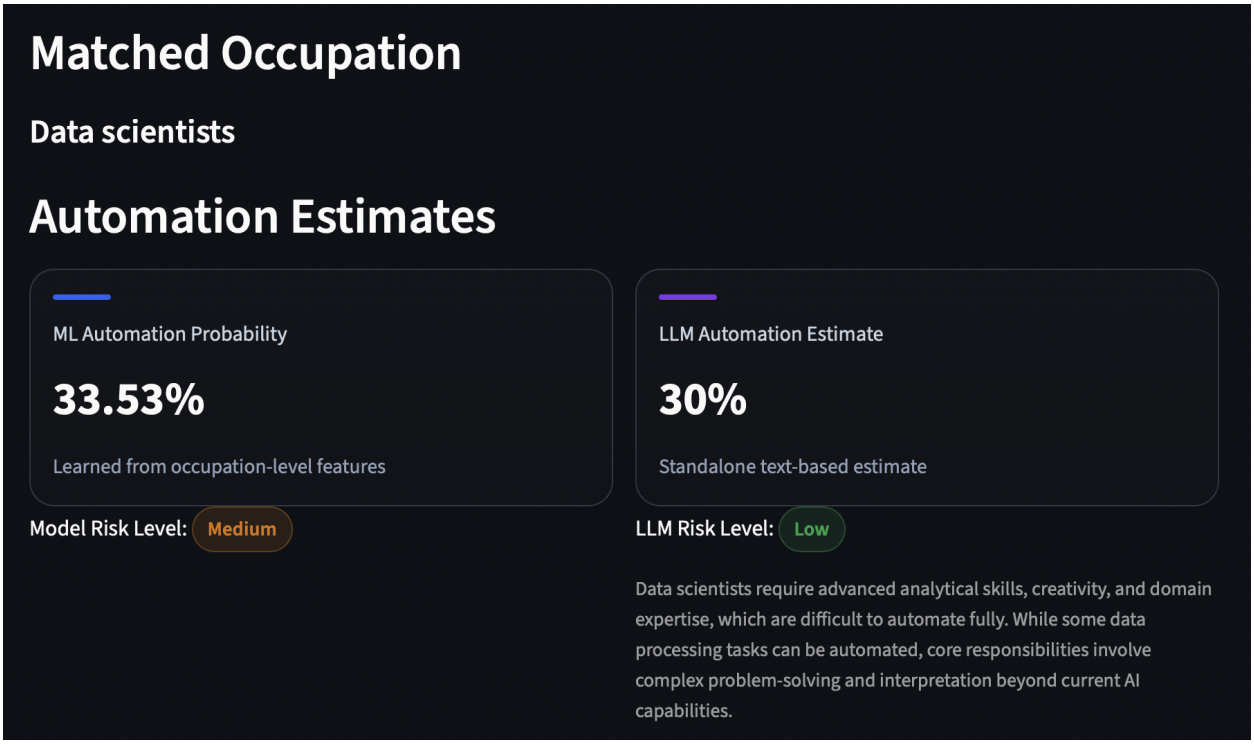

Figure 3: Main results view

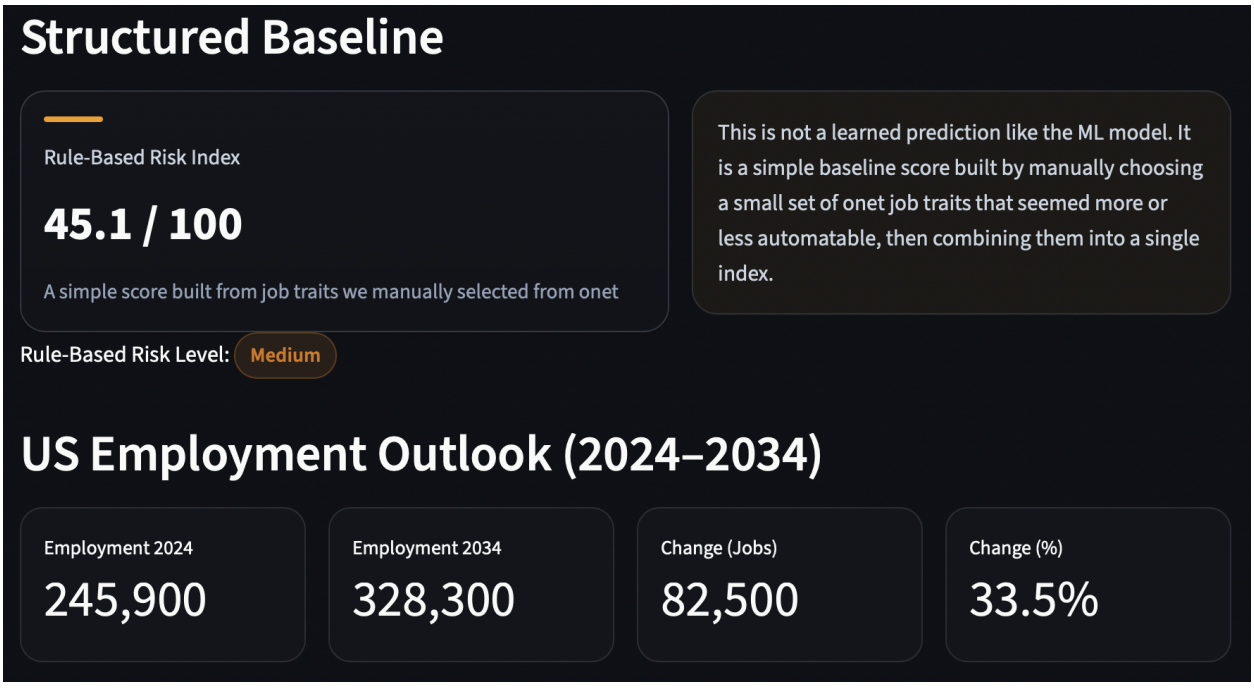

Figure 4: Baseline and employment outlook.

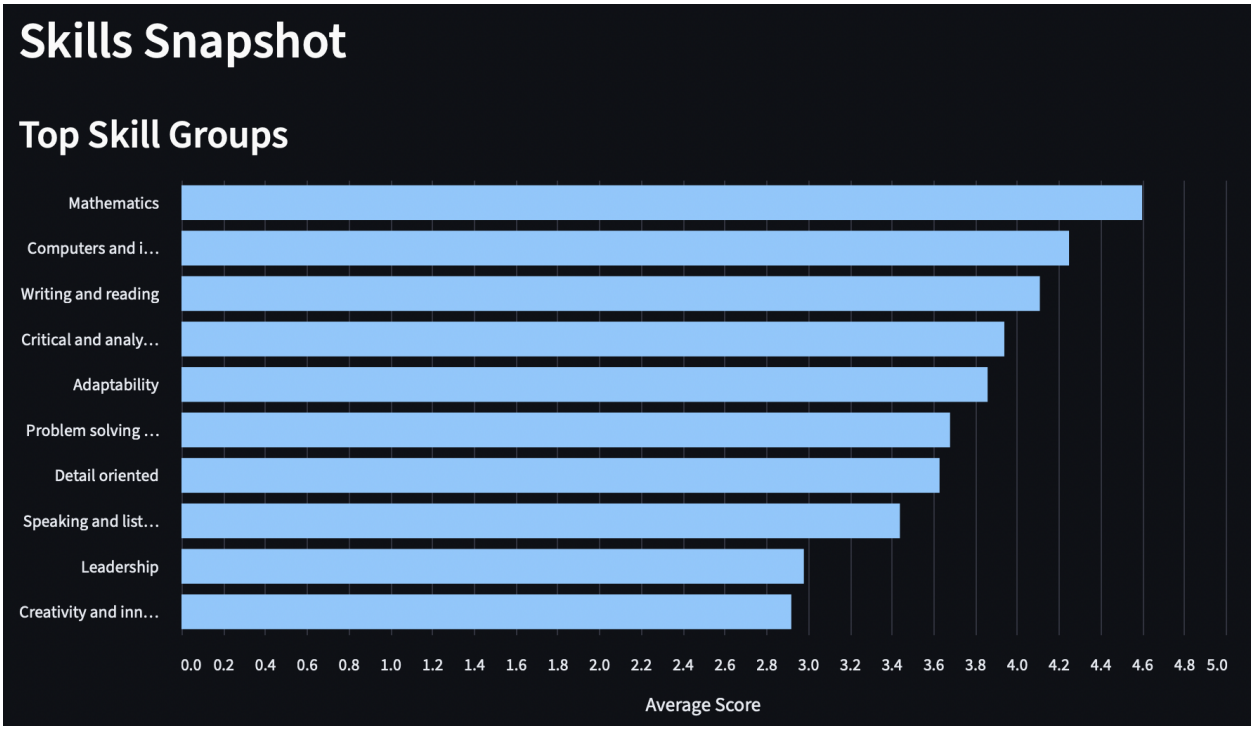

Figure 5: Skills snapshot.

9. Discussion

9.1 Interpretation of Results

The results in Table 8 show that the Random Forest regressor provided a stronger fit to the automation-risk target than the linear baseline. In practical terms, this suggests that occupation-level automation risk is not well captured by a purely linear relationship between skills and the target variable. The improvement in both error metrics and coefficient of determination indicates that the engineered feature space contains useful nonlinear structure that the Random Forest was better able to exploit.

At the same time, the results should be interpreted with some caution. An R^2 of 0.6012 indicates that the model explains a meaningful share of the variation in the target, but it does not mean that automation risk can be predicted perfectly from occupation-level features alone. This is reasonable given the complexity of the problem. Automation exposure depends not only on skills and task structure, but also on broader economic, organizational, and technological factors that are not fully captured in the dataset.

The comparison between the supervised model, the LLM-based estimate, and the rule-based baseline is also important. The Random Forest served as the main learned predictor because it was grounded

in labeled data and performed better than the linear baseline. The LLM-based estimate, by contrast, was useful as a flexible reasoning component, especially for handling user-entered job titles, but it was not designed or evaluated as the main supervised predictor. The rule-based baseline added interpretability by showing a simpler, manually structured view of automation exposure. Together, these components made the final system more informative than a single-score approach.

Rather than comparing the system to human performance, which is difficult to define for this task, it is more meaningful to compare it to existing static occupation-level approaches. In that context, the main contribution of the project is not only predictive performance, but also the ability to accept flexible job-title input, combine multiple perspectives on automation risk, and present the results together with employment outlook and skill-based context in one interactive application.

9.2 Limitations

1. **Data limitations:** The supervised learning pipeline depends on an external automation-risk dataset whose target values are themselves estimated occupation-level probabilities rather than directly observed outcomes. In addition, the final feature space is built from occupation-level structured data, which means it cannot fully capture differences in task mix across individual workers, firms, or industries.

2. **Model limitations:** The Random Forest model improves on the linear baseline, but it still explains only part of the variation in automation risk. The LLM-based component is useful for semantic matching and comparison, but its outputs may vary depending on prompting and API behavior. The rule-based baseline is interpretable, but it depends on manual design choices and is therefore limited in flexibility.

3. **Scope limitations:** The system was designed as an interactive decision-support and exploration tool rather than as a definitive forecast of job displacement. It estimates automation exposure at the occupation level and was not designed to model firm-level hiring behavior or long-term macroeconomic effects of AI adoption.

9.3 Ethical Considerations

This project raises several ethical considerations. Automation-risk estimates can be misinterpreted if they are treated as definitive predictions about whether a job will disappear. In practice, such estimates should be understood as approximate signals that help users think about changing job structure and exposure to automation.

The system also depends on structured labor data and externally estimated automation-risk labels, which may reflect existing assumptions or biases in how occupations are defined and evaluated. As a result, the outputs should be interpreted carefully and in context rather than as objective truth.

In addition, the LLM-based component may introduce variability because language-model outputs can be sensitive to phrasing. For this reason, the final application presents multiple views of automation risk instead of relying on a single estimate alone.

Finally, the project operates on occupation-level information rather than personal user data, which reduces privacy concerns. However, because tools like this can influence how users think about work and career stability, the results should be communicated responsibly.

10. Conclusion and Future Work

10.1 Summary of Contributions

This project addressed the problem of estimating job automation risk in a way that is more flexible and interpretable than static occupation-level scores alone. We built a system that accepts user-entered job titles, maps them to standardized occupations, and combines multiple views of automation exposure in one application.

The final approach centered on a supervised Random Forest regressor trained on occupation-level features engineered from O*NET-based data and an external automation-risk dataset. Alongside the supervised model, the system also included an LLM-based estimate and a rule-based baseline to provide comparison and additional context. The final application integrated these components with employment outlook and skill-based summaries in a deployed Streamlit interface.

The main result of the project was that the Random Forest regressor outperformed a linear baseline on the held-out test set, indicating that the engineered feature space captured meaningful nonlinear structure related to automation risk. More broadly, the project demonstrated that structured occupation-level modeling, semantic job-title matching, and interpretable comparison components can be combined into a single end-to-end system for exploring automation exposure.

10.2 Future Work

1. Expand the supervised learning pipeline with additional occupation and labor-market datasets to improve coverage and strengthen the target definition.
2. Improve the deployment architecture by adding stronger monitoring and more systematic testing of backend responses and edge-case job-title inputs.
3. Extend the application with richer interpretability features, such as side-by-side occupation comparison and broader employment-trend analysis.

References

Daron Acemoglu, David Autor, Jonathon Hazell, and Pascual Restrepo. Artificial intelligence and jobs: Evidence from online vacancies. *Journal of Labor Economics*, 40(S1):S293–S340, 2022.

Melanie Arntz, Terry Gregory, and Ulrich Zierahn. The risk of automation for jobs in oecd countries. Technical Report 189, OECD, 2016.

Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. Gpts are gpts: Labor market impact potential of large language models. *Science*, 384(6703):1306–1308, 2024.

Edward Felten, Manav Raj, and Robert Seamans. Measuring the exposure of occupations to artificial intelligence. *Strategic Management Journal*, 2021.

Appendix A: Full Pseudo-code Listing

A.1 Occupation Data Preparation Pipeline

Pseudo-code: End-to-End Occupation Data Preparation Pipeline (see Section 3)

Require: O*NET-based occupation dataset $\mathcal{D}_{onet}$, external automation-risk dataset $\mathcal{D}_{risk}$
Ensure: Final merged dataset $(\mathcal{D}_{train}, \mathcal{D}_{test})$ for supervised learning

Load $\mathcal{D}_{onet}$ and $\mathcal{D}_{risk}$ from source files
Normalize column names and occupation identifiers
Select relevant fields from $\mathcal{D}_{onet}$
Aggregate long-format O*NET records into one row per occupation
Pivot skill groups and O*NET element names into wide-format numeric columns
Retain employment-related variables as additional numeric features
Remove columns that are entirely missing or unusable for modeling
Merge engineered occupation-level table with $\mathcal{D}_{risk}$ using SOC code
if no direct SOC code match is found **then**
    Apply job-title matching as fallback
end if
Separate target variable y from feature matrix X
Split merged dataset into training and test sets using an 80/20 split
Save processed feature table and training artifacts for downstream modeling
return $(\mathcal{D}_{train}, \mathcal{D}_{test})$

A.2 Random Forest Training Pipeline

Pseudo-code: Random Forest Training Pipeline (see Section 4)

Require: Merged dataset $\mathcal{D}$ with features X and target y
Ensure: Trained model pipeline $\mathcal{M}^*$

Load $\mathcal{D}$ from disk
Remove rows with missing target values
Select numeric feature columns
Exclude identifier and label columns
Remove columns with no observed values
Split (X, y) into training and test sets using an 80/20 split
Define baseline pipeline:
    median imputation $\rightarrow$ Linear Regression
Define Random Forest pipeline:
    median imputation $\rightarrow$ Random Forest Regressor

Fit baseline pipeline on training data
Predict on test data
Compute MAE, RMSE, and R^2
Fit Random Forest pipeline on training data
Predict on test data
Compute MAE, RMSE, and R^2
Extract feature importances from Random Forest
Save trained pipeline, predictions, and feature importances
return $\mathcal{M}^*$

Appendix B: Individual Contribution Statement

Rayan Hassan

I designed and implemented the AWS-based backend pipeline that powers the LLM component of the system. I built the full data flow using AWS services, including API Gateway for handling incoming requests, AWS Lambda for backend processing, and S3 for storing and retrieving the O*NET skills dataset and precomputed feature data.

For each user-input job title, I designed a dynamic pipeline that retrieves relevant data from S3, analyzes it to identify matching occupations, and extracts their associated skills and employment information. I then structured this data and sent it to the OpenAI platform, where I developed and prompt-engineered an Assistant to generate automation risk estimates and the reasoning behind it.

Elif-Selma Yilmaz

My main contributions focused on the supervised learning pipeline. I designed the preprocessing and feature-engineering workflow that transformed the O*NET-based occupation data from long format into an occupation-level feature table suitable for modeling. This included cleaning the structured data, aggregating skill and employment variables, merging the engineered features with the external automation-risk dataset, and preparing the final training and test split.

I then implemented and evaluated the main machine learning pipeline. I trained the Random Forest regressor, compared it against a linear baseline, analyzed the resulting metrics, and integrated the saved model into the Streamlit application for local inference. I also added lightweight automated tests with `pytest` for key preprocessing and model-training utilities and helped organize the final GitHub repository for the completed system.